S.No: 9	Exp. Name: ***Breadth First Search (BFS)***	Date:2026-07-10

Aim:

Write a program to implement Breadth First Search (BFS) graph traversal methods.

Input Format:

- The first line contains an integer N, the number of vertices in the graph.
- The second line contains an integer E, the number of edges in the graph.
- The next E pairs of lines contain two integers each:
  The first line of each pair contains an integer s representing the source vertex.
  The second line of each pair contains an integer d representing the destination vertex.
- The last line contains an integer v, which is the starting vertex for the BFS traversal.

Output Format:

- The program will output the vertices of the graph, each on a new line, in the order they are visited by the BFS traversal, starting from the vertex v.

Note:

- Driver code is already provided to you in the editor, your task is to fill in the required code.
- Refer to visible test cases for better understanding.

Source Code:

GraphsBFS.c

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 99

struct node {
    int vertex;
    struct node* next;
};
typedef struct node* GNODE;

GNODE graph[20];
int visited[20];
int queue[MAX], front = -1, rear = -1;
int n;

void insertQueue(int vertex) {
    if (rear == MAX - 1)
        printf("Queue Overflow.\n");
    else {
        if (front == -1)
            front = 0;
        rear++;
        queue[rear] = vertex;
    }
}

int isEmptyQueue() {
    return (front == -1 || front > rear);
}
```

```
int deleteQueue() {
    if (isEmptyQueue()) {
        printf("Queue Underflow\n");
        exit(1);
    }
    return queue[front++];
}

void BFS(int v) {
    // Write your code here

    int current;
    GNODE temp;

    // Mark the starting node as visited and enqueue it
    visited[v] = 1;
    insertQueue(v);

    // Continue processing while there are elements in the queue
    while (!isEmptyQueue()) {
        current = deleteQueue();

        // Print the visited node followed immediately by a newline
        printf("\n%d", current);

        // Traverse all connected neighbors in the adjacency list
        temp = graph[current];
        while (temp != NULL) {
            int adjVertex = temp->vertex;

            // If the neighbor hasn't been visited yet, mark it and enqueue it
            if (!visited[adjVertex]) {
                visited[adjVertex] = 1;
                insertQueue(adjVertex);
            }
            temp = temp->next;
        }
        }



}

void main() {
    int N, E, s, d, i, v;
    GNODE p, q;

    printf("Enter the number of vertices: ");
    scanf("%d", &N);
    n = N;

    for (i = 0; i < N; i++) {
        graph[i] = NULL;
        visited[i] = 0;
    }
```


ID: 25070521290

```
    printf("Enter the number of edges: ");
    scanf("%d", &E);
    for (i = 1; i <= E; i++) {
        printf("Enter source: ");
        scanf("%d", &s);
        printf("Enter destination: ");
        scanf("%d", &d);

        q = (GNODE)malloc(sizeof(struct node));
        q->vertex = d;
        q->next = NULL;

        if (graph[s] == NULL) {
            graph[s] = q;
        } else {
            p = graph[s];
            while (p->next != NULL)
                p = p->next;
            p->next = q;
        }
    }

    printf("Enter Start Vertex for BFS: ");
    scanf("%d", &v);
    printf("BFS of graph: ");
    BFS(v);
    printf("\n");
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the number of vertices:
5
Enter the number of edges:
5
Enter source:
1
Enter destination:
2
Enter source:
1
Enter destination:
4
Enter source:
4
Enter destination:
2
Enter source:
2

Enter destination:
3
Enter source:
4
Enter destination:
5
Enter Start Vertex for BFS:
1
BFS of graph:
1
2
4
3
5

Test Case - 2
User Output
Enter the number of vertices:
4
Enter the number of edges:
3
Enter source:
1
Enter destination:
2
Enter source:
2
Enter destination:
3
Enter source:
3
Enter destination:
4
Enter Start Vertex for BFS:
2
BFS of graph:
2
3
4